SSL-PROJECT-REPORT
Snake Stack

Shiven Toshniwal Rishabh Iyer

May 2026

Contents

1 External libraries used with purpose **2**

2 Description of all features implemented **2**
2.1 Gameplay 2
2.2 Admin 3

3 Directory structure & how the 3 layers communicate **4**
3.1 Directory structure 4
3.2 How layers communicate 4
3.3 Format for history.txt 4

4 Usage **5**

5 Hurdles faced and how we overcame them **5**

6 What we would do in 10 extra hours **5**

1 External libraries used with purpose

- Bootstrap 5 front end framework - For Styling [1]
- Math.random - For random generation of food items on the grid [5]
- Math.floor - For random selection of coordinates as well as random selection of fruits [4]
- Flask - Provides web server and routing capabilities [2] [3]
- datetime - Formats dates for the history.txt log
- os - Handles directory paths for serving static files

2 Description of all features implemented

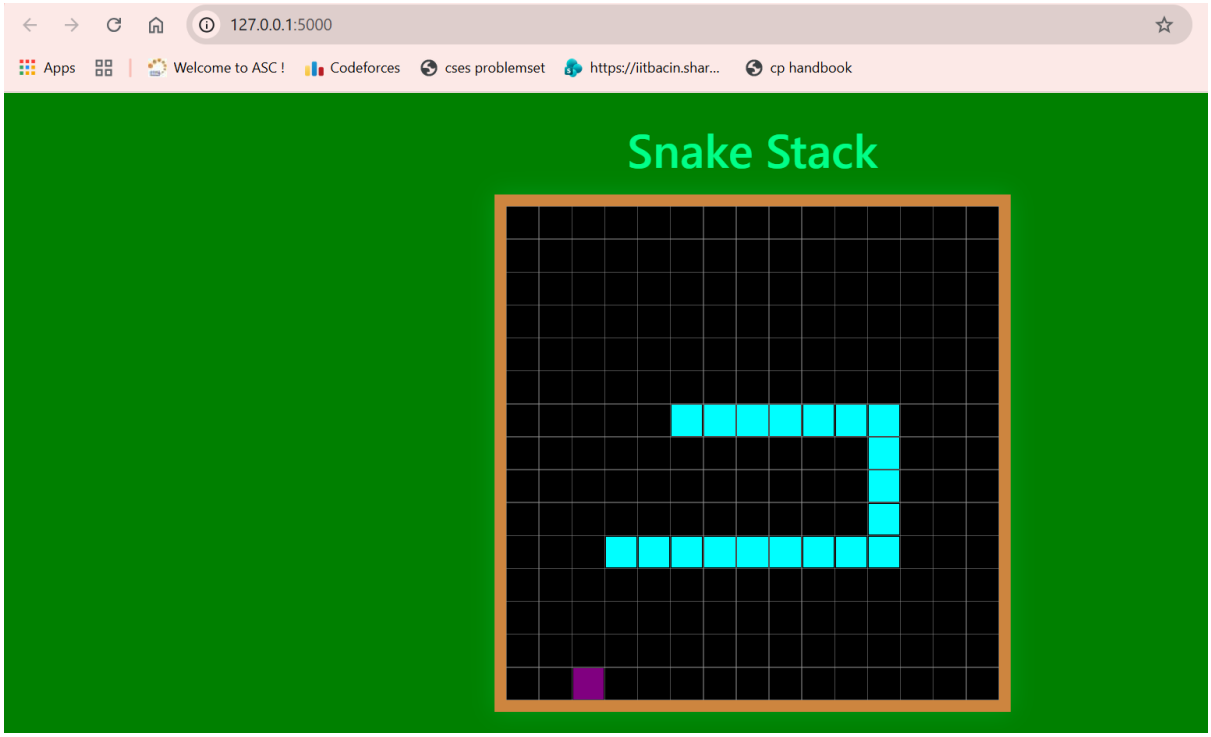
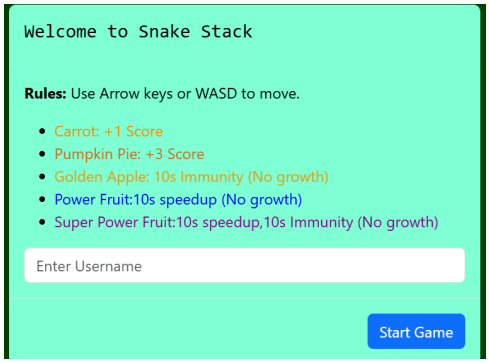


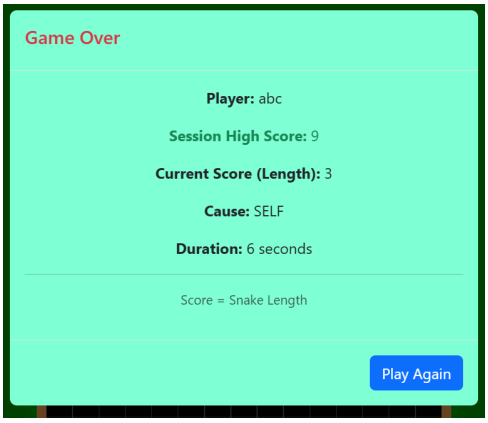
Figure 1: Ongoing game

2.1 Gameplay

- Welcome modal on starting game with start button and game information and asks for username which if not entered leads to display of alert message. (2a)
- Storing snake as an array of pairs of coordinates.
- Taking keyboard input to change position of head and popping last element of array.
- Storing food items as array of pairs of coordinates.



(a) Game Start



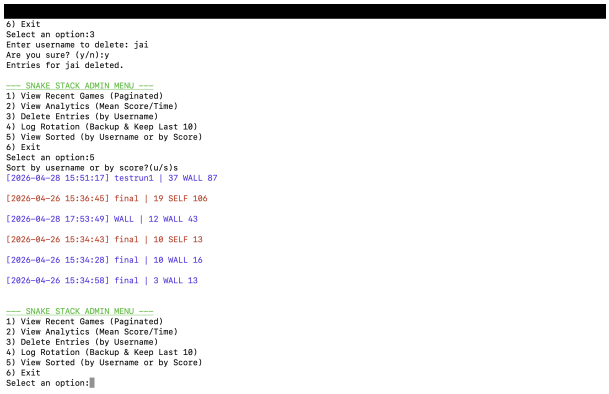
(b) Game Over

Figure 2: User Interface

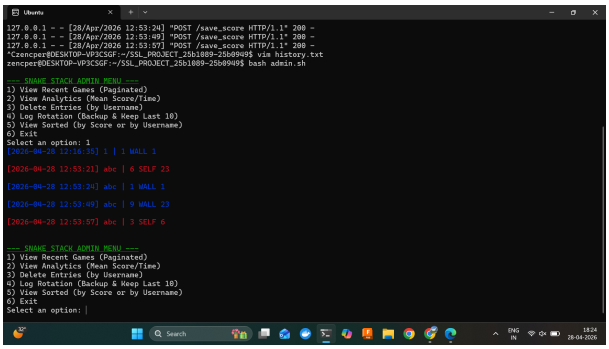
- Every 200ms, the boxes corresponding to coordinates of snake and foods are colored.(1)
- Snake dies when the coordinates of head either coincide with another element of snake array or when it leaves the grid.
- 5 different types of fruits that the snake can feed on. One food item instantly appears when the previous is eaten.
- Incrasing length by retaining last "food.value" number of elements in snake array.
- Setting a 10s immunity timer in which the snake does not die on either hitting the wall or itself.
- Setting a 10s speed up by reducing the time interval between successive renderings of the board.
- Displays game-play statistics of current session.(2b)

2.2 Admin

- Checking for existence and emptiness of history.txt.
- Display menu driven interface
  - Viewing Recent games
  - Viewing analytics (Mean score/time)
  - Deleting entries by username
  - Log rotation in which history.txt is truncated to last 10 entries and backup is created in the form of .tar.gz file.
  - Sorting by username or score (3a)
  - Exit option
- Handling invalid input and displaying error message.
- Using ANSI escape codes for colored output. (3b)



(a) Sorting

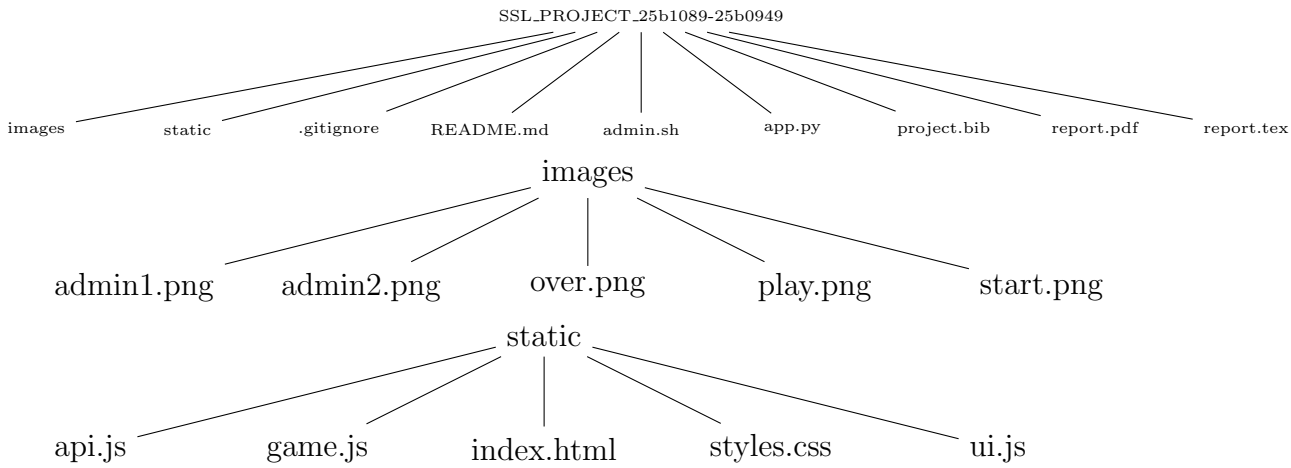


(b) Recent games

Figure 3: Admin Features

3 Directory structure & how the 3 layers communicate

3.1 Directory structure



Also a Makefile exists in SSL_PROJECT_25b1089-25b0949

3.2 How layers communicate

At first, the user interacts with JavaScript front end framework. When the game ends, the front end sends the final score to Flask Server using fetch API. Request is POST with a JSON body. Inside app.py, the score is parsed, validated and appended to history.txt with timestamp. Managing history.txt and backup files by admin.sh. We have used bash commands like sed, awk, grep, cut, sort and case to create a menu driven interface with different options.

3.3 Format for history.txt

[timestamp] name | score cause duration
Chosen because with this format, sorting becomes easier and since game data is space separated, it can be handled by awk.

4 Usage

- Running game on server

  Type 'python3 app.py' on command line.
  Navigate to http://127.0.0.1:5000 on browser.
  Enter username and press Start Game.
  Can use arrow keys or WASD for snake motion.
  Press Play Again to restart if game over

- Running admin.sh

  Type 'bash admin.sh' on command line.
  Navigate menu by entering number corresponding to required option.
  Where valid, enter required suboptions out of those visible

5 Hurdles faced and how we overcame them

- Adjusting the display size of the grid and modal - By trial and error.
- Adjusting timers for changing speed of snake and setting immunity - Disabling existing loop and setting timer for running a new loop temporarily.
- Ensuring proper parsing and sorting of data in history.txt. For example, if username matches WALL or SELF - By using proper delimiters as per history.txt's format.
- Ensuring that a fruit does not generate on snake's body - Keeping in mind that the randomly generated fruit appears on the grid excluding all the coordinates inside snake array.

6 What we would do in 10 extra hours

- Improve the graphics of both the snake and fruits.
- An option for the user to vary speed of snake, vary the types and numbers of fruits generated at a time.

References

[1] The Bootstrap Authors. Bootstrap5. https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css.

[2] *Flask documentation*. https://flask.palletsprojects.com/en/stable/.

[3] Real Python. Flask tutorials. https://realpython.com/tutorials/flask/.

[4] W3Schools. Js math floor() method. https://www.w3schools.com/jsref/jsref_floor.asp.

[5] W3Schools. The js random. https://www.w3schools.com/js/js_random.asp.